

Yazılım Test Mühendisliği ve Yapay Zeka Destekli API Otomasyonu

Rest Assured ile Otomatik Regresyon Testi Projesi

BOLU ABANT İZZET BAYSAL ÜNİVERSİTESİ — BİLGİSAYAR MÜHENDİSLİĞİ

SEDEFNAZ DURUKAN

243405046

Made with GAMMA

Yazılım Test Mühendisliği Nedir?

Yazılım Testinin Amacı

Ürünün kalitesini artırmak, hataları (bug) son kullanıcıya ulaşmadan yakalamak ve maliyetleri düşürmek.

Geleneksel Test Yaklaşımları

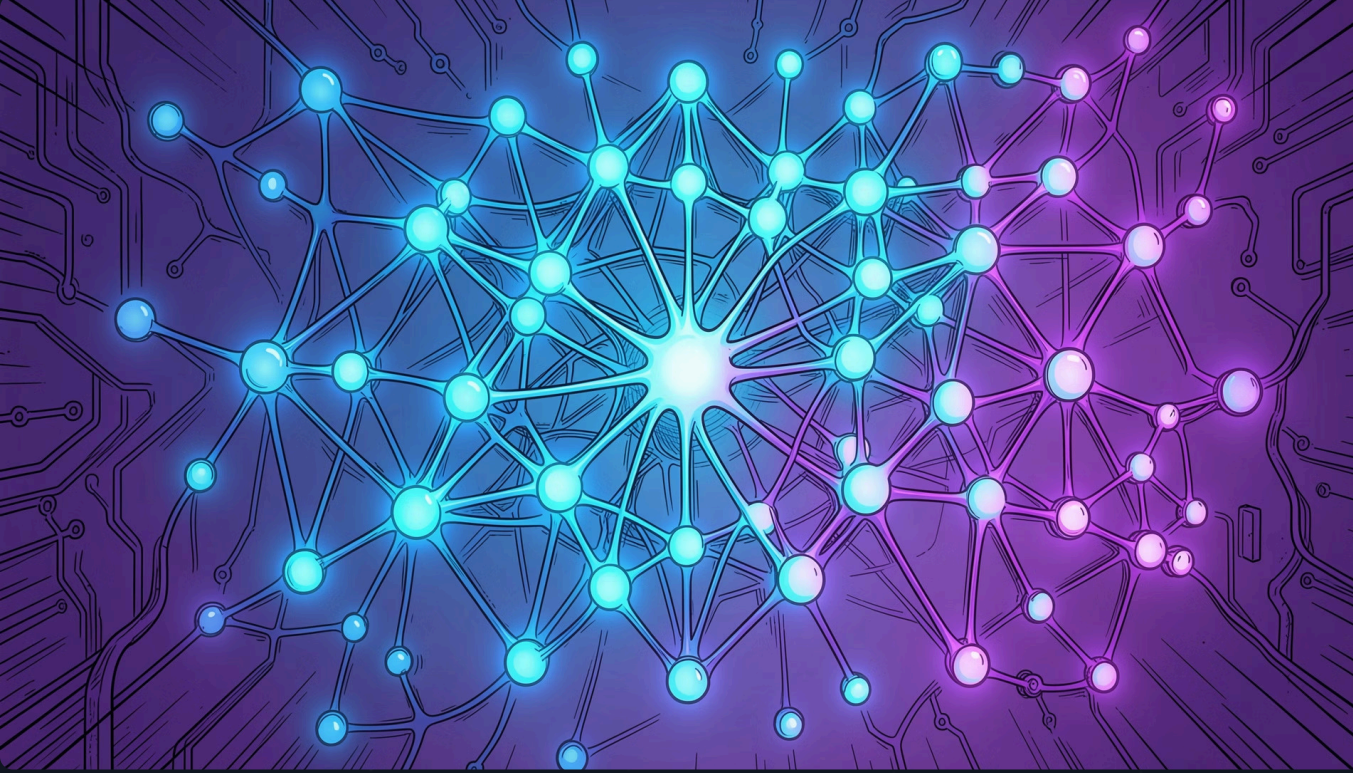
- Manuel Test — Zaman alıcı, insan hatasına açık
- Klasik Otomasyon — Kod yazım maliyeti yüksek, kırılgan yapılar

Günümüzdeki İhtiyaç

Hızlı canlıya çıkış (CI/CD süreçleri) ve kesintisiz Regresyon Testleri.



Yapay Zeka (AI) Destekli Yazılım Test Mühendisliği



Test Dünyasında Yapay Zeka Devrimi:
AI, test mühendislerinin yerini almak için değil, süreçleri hızlandırmak için bir yardımcı olarak konumlanmaktadır.

→ Kendi Kendini İyileştiren (Self-Healing) Testler

UI değişikliklerinde elementleri otomatik bulma.

→ Akıllı Test Senaryosu Oluşturma

Gereksinim dokümanlarından (PRD) yapay zekanın otomatik test senaryoları üretmesi.

→ Kod Analizi ve Hata Tahmini

Hangi kod değişikliğinin nerede hata çıkarabileceğini tahmin eden algoritmalar.

Projede Kullanılan Teknolojiler



Programlama Dili

Java — Nesne yönelimli, güvenilir altyapı



Build Aracı

Maven — Bağımlılık yönetimi ve pom.xml mimarisi



Test Framework

JUnit 5 — Modern test anotasyonları ve raporlama



Kütüphane

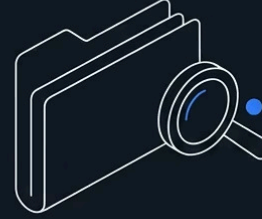
Rest Assured — REST API'leri test etmek için geliştirilmiş en popüler Java kütüphanesi



Ödev Projesi — Test Senaryoları

Hedef

Gerçek dünya senaryolarına uygun olarak **GitHub REST API** uç noktalarını doğrulamak.



GET İstekleri

Veri çekme senaryoları



Durum Kodları

200 ve 404 gibi kodların kontrolü



Yanıt Doğrulama

JSON parametrelerinin kontrolü



Performans Zamanlaması

Milisaniye bazında yanıt süreleri

1

GET İstekleri

Başarılı veri çekme senaryoları.

2

Status Code Kontrolleri

API'den beklenen durum kodlarının (200, 404 vb.) doğrulanması.

3

Response Body Doğrulaması

Dönen JSON metni içindeki parametrelerin tam kontrolü.

4

Performans (Süre) Kontrolü

İsteklerin x milisaniyenin altında yanıt dönmesinin ölçülmesi.

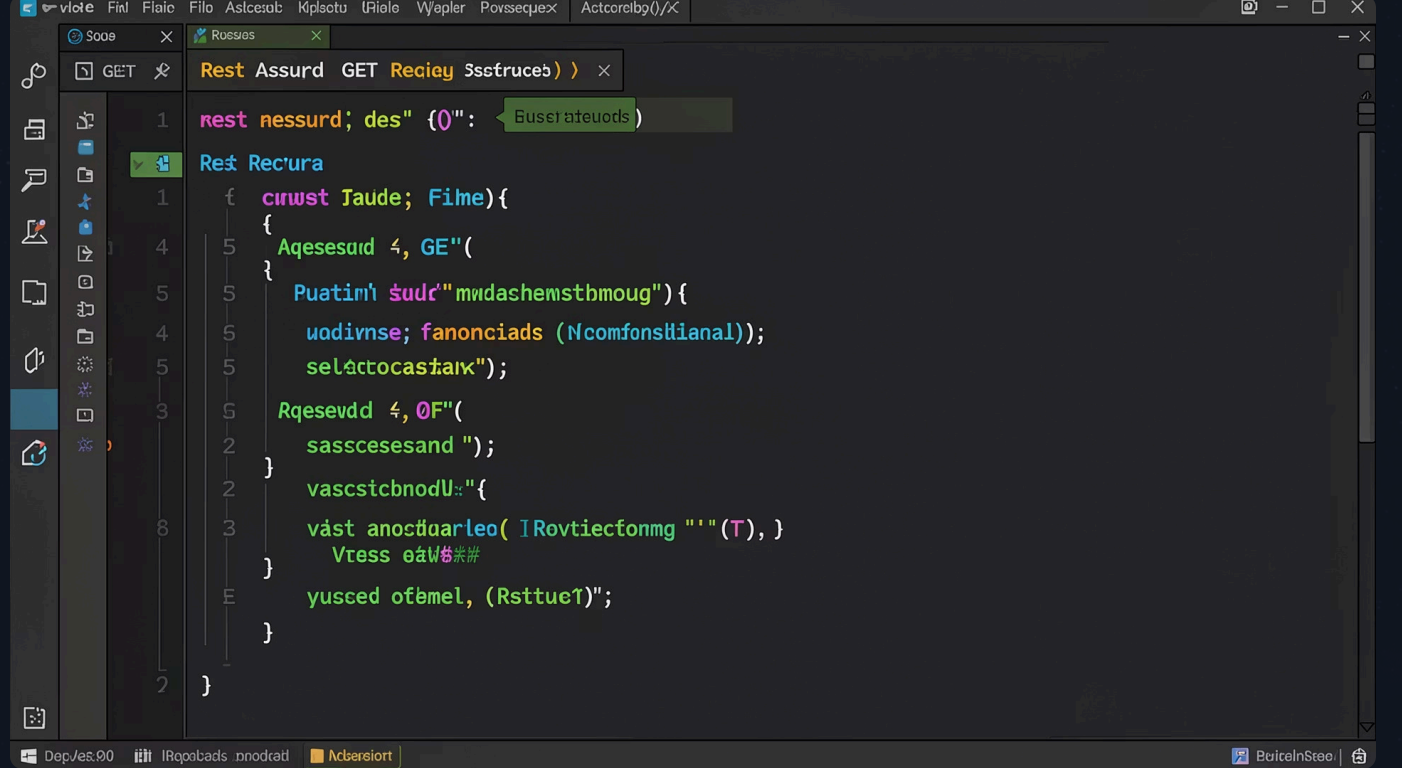
Kod Yapısı — Olumlu (Success) Senaryo

Metot: `testGitHubRepoGetir()`

Yapılan İşlem:

`https://api.github.com/repos/junit-team/junit5` adresine GET isteği gönderilir.

Doğrulamalar (Assertions)



```
1 rest nassurd; des" (0": Busei ateuods )
2
3 Rest Recura
4 {
5   ciwst Taude; Filme){
6   {
7     Aqesesard 4, GE"(
8     {
9       Puatiml sudr" mwdashemsthmoug"){
10      uadiwse; fanonciads (Ncomfonstlianal));
11      selactocastark");
12    }
13    Rqesevdd 4, 0F"(
14      sasscesesand ");
15    vascsticbnodU: "{
16      vast anostuarleo( IRovtiecfonmg ""(T), )
17      Vress ealw6##
18    }
19    yusced ofemel, (Rsttue1");
20  }
21 }
```

☐ Durum kodunun **200 OK** olduğu kontrol edilir.

☐ Repository isminin (name) "**junit5**" olduğu ve private alanının **false** olduğu teyit edilir.

☐ Yanıt süresinin belirlenen süre sınırının altında kaldığı ölçülür.

testGitHubRepoBulunamadi()

Amaç: Sistemlerin sadece doğru girdilerde değil, hatalı girdilerde de doğru tepkiyi (hata yönetimini) verdiğini doğrulamak.

Doğrulamalar (Assertions)

Hata mesajının JSON içeriğindeki karşılığı test edilir.

Test Sonuçları — Regresyon Başarısı

0

Failure

Tüm testler başarıyla geçti

0

Error

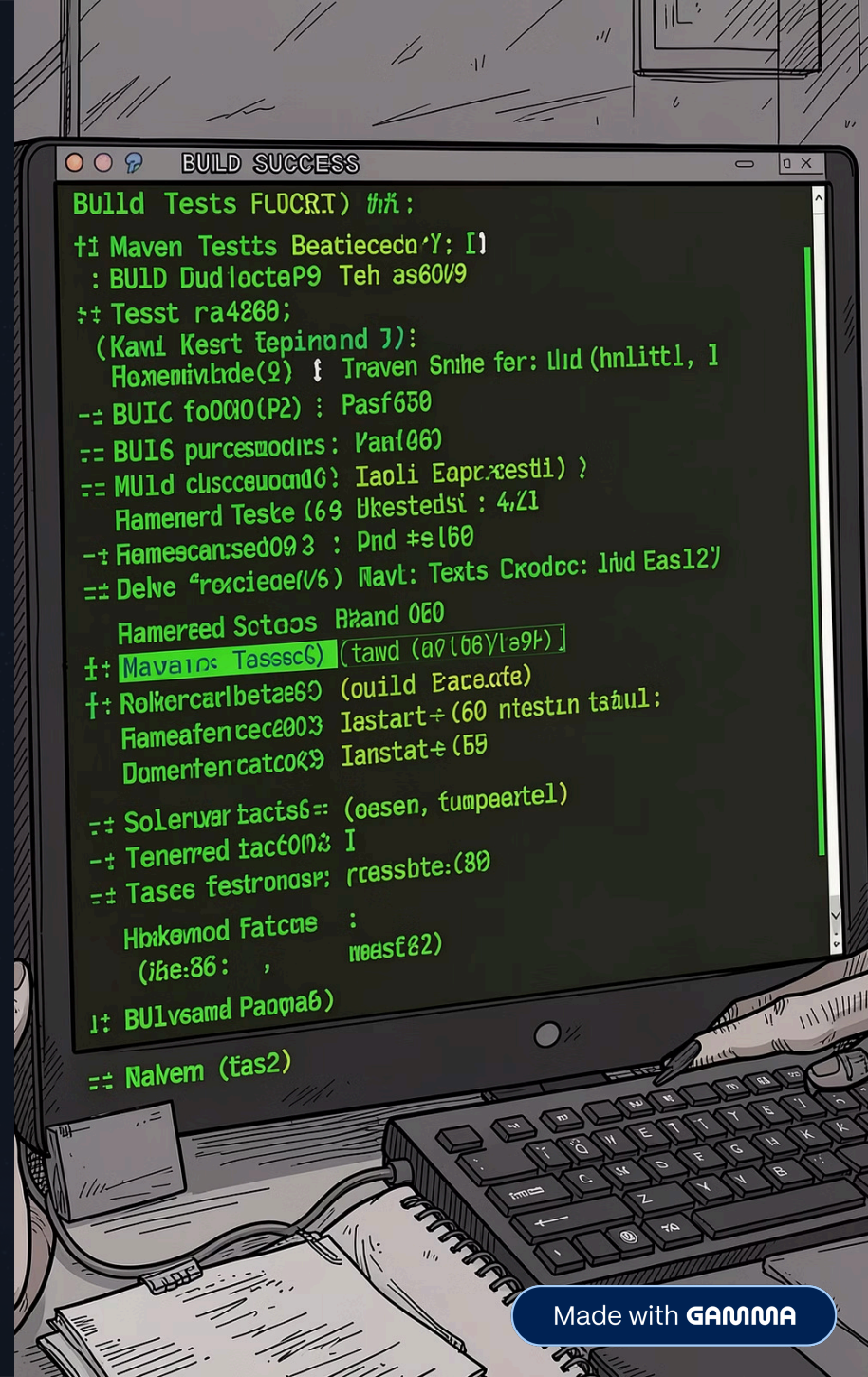
Hiçbir hata oluşmadı

100%

Başarı Oranı

Tüm senaryolar doğrulandı

Maven entegrasyonu sayesinde `mvn clean test` komutuyla tüm test süreci otomatikleştirilmiştir. Proje sıfır hata (Failures: 0, Errors: 0) ile başarılı bir şekilde derlenmiş ve koşturulmuştur. Bu otomasyon altyapısı, projeye yeni bir özellik eklendiğinde API'lerin kırılıp kırılmadığını saniyeler içinde ölçmemizi sağlar.





Kaynak Kodlar

Projenin kaynak kodları,
[README.md](#) dokümantasyonu
ve bu sunum dosyası **Public**
GitHub reposuna yüklenmiştir.



Dinlediğiniz için teşekkür
ederim.